

STEADY

Clinician Thoughts, Clinical Memory and Session Prep

Implementation-Ready Engineering Specification, v2.0

Repository-grounded coding handoff | Reviewed against tprawlins-lang/Emdr | branch:
claude/gifted-keller-501y5d | September 2026

Build directive

Add a voice-first clinician thinking layer to the existing clinician member workspace. A clinician records natural thoughts, Steady transcribes and organizes them, the clinician reviews the result, approved items become source-backed clinical memory, and Steady uses that memory for threads, Session Prep, and patient-scoped retrieval.

Non-negotiable product rule

Steady remembers what happened. Steady organizes what the clinician says. Steady suggests connections. The clinician decides what those connections mean.

This version is written so a coding agent can work phase by phase without inventing product behavior. Where the repository already defines a pattern, this handoff uses it. Where the repository is still in transition, this handoff marks the dependency and gives the intended target state.

1. Executive Build Snapshot

- **Primary surface:** `src/app/clinician/member/[id]/page.tsx` and the shared `PersonShell` patient workspace.
- **Current data runtime:** SQLite through `better-sqlite3` in `src/lib/db.ts`. Postgres and RLS exist as a migration target, but are not the active runtime on the reviewed branch.
- **History model:** ADR 0010 has shipped event dual-write and replay verification. Event-authoritative cutover is not yet complete. New work must use the spine and must not create another untracked write path.
- **Tenancy:** tenant fields, person/account split, repository layer, and Postgres RLS building blocks exist. ADR 0011 states end-to-end call-site migration is still incomplete. New feature code must use tenant-scoped access from day one.
- **AI:** ADR 0012 requires every model call to pass through `src/lib/ai-gateway/`. On the reviewed branch, this remains a required architectural dependency rather than a completed invariant. Do not add another direct provider call.
- **Existing clinician design language:** the member overview already uses deterministic evidence-backed summaries, explicit review states, source links, and a right-rail action model. The new feature should extend those conventions rather than introduce a separate visual system.
- **Recommended delivery:** six gated phases. Capture and transcript first, then structured organization, then threads, Session Prep, retrieval, and formal-note bridge.

Definition of implementation-ready: Code should be able to start with Phase 1 using the schemas, commands, events, UI states, failure behavior, tests, and file map below. Product questions that can change patient safety or record meaning are resolved here rather than left to the coding agent.

2. Repository Facts That Shape This Feature

Current pattern	Observed repo location	What it means for this build
Next.js App Router	<code>src/app/...</code>	Use server pages/actions and route handlers consistent with the existing app.
Clinician patient workspace	<code>src/app/clinician/member/[id]/page.tsx</code>	Record Thoughts belongs inside the existing person workspace, not a stand-alone clinician app.
Patient record view	<code>src/app/clinician/member/[id]/record/page.tsx</code>	Thread evidence and source drill-down should link into this record/timeline surface.
Clinical read modules	<code>src/lib/clinical/*</code>	Add thought/thread/session-prep read models here rather than mixing clinical logic into page components.
SQLite schema and boot migration	<code>src/lib/db.ts</code>	Add SQLite tables through the existing schema/migration model now; mirror to Postgres schema at the same time.
Longitudinal event spine	<code>src/lib/spine.ts</code> , <code>src/lib/projections.ts</code>	Mutations must append registered events and project state. Do not create a side-channel notes table with direct writes only.
Tenant repository	<code>src/lib/repository.ts</code>	New reads/writes must require <code>TenantContext</code> . Do not copy the legacy direct-query pattern from old call sites.
Audit logging	<code>src/lib/audit.ts</code>	Record user-visible clinical changes and access to sensitive artifacts according to policy.
Clinical review conventions	<code>src/lib/clinical/review-notes.ts</code>	Typed statuses, clear ownership, audit stamping, and explicit workflow state already exist.
AI architecture decision	<code>docs/adr/0012-ai-gateway.md</code>	Extraction, linking, Session Prep generation, and Ask Steady all become registered gateway tasks.

Important sequencing note: ADR 0010 says event append is not yet the only authoritative write path. ADR 0011 says tenant enforcement exists as building blocks but all product call sites have not yet moved behind it. This feature should be implemented to the target architecture, even while legacy areas are still migrating.

3. Clinician UX, End to End

Primary goal: A clinician should be able to capture a useful post-session thought with no filing work. In the normal path, the clinician speaks, reviews the organized result, and saves in roughly 60 to 90 seconds total.

The patient workspace should expose a persistent **Record Thoughts** action. Do not ask the clinician to choose a note type, category, diagnosis, tag, or folder before recording.

The interaction has six states: capture, transcription, organization, review, connection review, saved.

```
PATIENT WORKSPACE
|
+-- Record Thoughts
|   |
|   +-- Recording
|   |   Record / Pause / Resume / Cancel / Done
|   |
|   +-- Processing
|   |   Upload -> Transcribe -> Extract
|   |
|   +-- Review
|   |   Original transcript
|   |   "Steady organized your thoughts"
|   |   Edit / Remove per card
|   |   Approve all / Save thoughts
|   |
|   +-- Candidate connections
|   |   Existing thread candidate
|   |   Connect / Not related
|   |
|   +-- Saved
|       Thought available in Thoughts
|       Approved items feed patient memory
|       Follow-ups feed Session Prep
```

3.1 Recording screen

- Use a compact modal or sheet that keeps the clinician anchored to the patient context.
- Display patient name at the top so the clinician can immediately catch a wrong-person error.
- Controls: Record, Pause, Resume, Done, Cancel.
- Show elapsed time and a clear live recording state. No decorative AI language.
- Recommended soft duration guidance: 30 seconds to 3 minutes. Do not hard stop at 3 minutes unless storage or transcription policy requires it.
- On Cancel, require a second action only if audio has already been captured.
- If navigation or network failure occurs after capture begins, preserve a recoverable temporary artifact when the security model permits it.

3.2 Review screen

- Show the transcript first or make it one click away. The clinician must always be able to see what Steady heard.
- Show organized cards below it. Each card carries a type, concise text, provenance label, and Edit / Remove controls.
- Primary action: **Save Thoughts**. Do not make the clinician confirm every card one by one in the ordinary path.
- AI-suggested relationships to existing threads appear after the main review. They are not auto-accepted.
- If the transcript is corrected before save, rerun extraction against the corrected transcript and keep the original transcript version in history.

4. Clinical Memory Taxonomy

Type	Meaning	Can AI create candidate?	Requires clinician approval?
observation	Clinician's observed presentation or behavior	Yes	Yes
symptom	Symptom or symptom change stated by clinician/patient	Yes	Yes
person_relationship	Named or described relationship relevant to care	Yes	Yes
event	Life or care event	Yes	Yes
intervention	Intervention used or discussed	Yes	Yes
intervention_response	Observed or reported response	Yes	Yes
treatment_target	Potential or active target	Yes	Yes
follow_up	Something clinician wants to revisit or do	Yes	Yes
goal	Goal stated or adopted	Yes	Yes
theme	Clinician-recognized recurring theme	Yes	Yes
clinician_hypothesis	Speculative clinician thinking	Yes	Yes, remains hypothesis
clinician_uncertainty	Explicit uncertainty, readiness question, unresolved judgement	Yes	Yes, never flattened into fact

Rule: The system must preserve epistemic status. “I think this may connect to abandonment” is not equivalent to “abandonment is an active patient theme.” “I am not sure she is ready” is not equivalent to “patient is not ready.”

5. Three-Layer Memory Model

Layer	Stores	Mutation rule	Example
1. Source	Audio metadata, transcript versions, formal notes, check-ins, assessments	Append or supersede, do not silently overwrite	Clinician recording and transcript
2. Clinical memory	Approved small structured items	Corrections append a superseding event	Follow-up: check sleep next session
3. Intelligence	Patterns, candidate links, generated summaries, retrieval answers	Never becomes clinical fact without an explicit clinician action	Possible link between sleep disruption and family stress

Do not maintain one AI-written master patient summary. Build small, source-backed objects and create views from them. This keeps provenance inspectable and supports point-in-time reconstruction.

6. Proposed Database Schema

The reviewed branch runs SQLite today. Add these tables to the SQLite schema and boot migration path in `src/lib/db.ts`, and mirror them in `scripts/pg-schema.sql` so the future Postgres cutover does not create schema drift. The SQL below is the logical schema. Use SQLite-compatible storage types now, with equivalent Postgres types later.

```
-- 1. One clinician thought capture
CREATE TABLE clinician_thoughts (
  id TEXT PRIMARY KEY,
  tenant_id TEXT NOT NULL,
  person_id TEXT NOT NULL,
  clinician_person_id TEXT NOT NULL,
  status TEXT NOT NULL CHECK (
    status IN ('capturing','processing','review','saved','discarded','failed')
  ),
  audio_storage_key TEXT,
  audio_retention_policy TEXT NOT NULL DEFAULT 'org_default',
  audio_deleted_at TEXT,
  current_transcript_id TEXT,
  source_session_id TEXT,
  recorded_at TEXT NOT NULL,
  saved_at TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);
```

```
-- 2. Versioned transcripts
CREATE TABLE clinician_thought_transcripts (
  id TEXT PRIMARY KEY,
  tenant_id TEXT NOT NULL,
  person_id TEXT NOT NULL,
  thought_id TEXT NOT NULL,
  version INTEGER NOT NULL,
  transcript_text TEXT NOT NULL,
  transcript_hash TEXT NOT NULL,
  provider TEXT,
  provider_model TEXT,
  language TEXT,
  confidence_json TEXT,
  created_by TEXT NOT NULL CHECK (
    created_by IN ('transcription_service','clinician')
  ),
  created_at TEXT NOT NULL,
  UNIQUE(thought_id, version)
);
```

```
-- 3. Extracted and approved memory items
CREATE TABLE clinical_memory_items (
  id TEXT PRIMARY KEY,
  tenant_id TEXT NOT NULL,
  person_id TEXT NOT NULL,
  source_thought_id TEXT,
  source_transcript_id TEXT,
  source_span_json TEXT,
  item_type TEXT NOT NULL,
  statement_class TEXT NOT NULL CHECK (
    statement_class IN (
      'clinician_observation','patient_report',
      'clinician_hypothesis','clinician_uncertainty'
    )
  ),
  normalized_label TEXT,
  display_text TEXT NOT NULL,
  status TEXT NOT NULL CHECK (
    status IN ('candidate','approved','rejected','superseded')
  ),
  approved_by TEXT,
  approved_at TEXT,
  supersedes_id TEXT,
  created_at TEXT NOT NULL
);
```

```

-- 4. Longitudinal threads
CREATE TABLE clinical_threads (
  id TEXT PRIMARY KEY,
  tenant_id TEXT NOT NULL,
  person_id TEXT NOT NULL,
  thread_type TEXT NOT NULL,
  canonical_label TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('active','resolved','archived')),
  created_by TEXT NOT NULL CHECK (created_by IN ('clinician','system')),
  first_seen_at TEXT,
  last_seen_at TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE clinical_thread_memberships (
  id TEXT PRIMARY KEY,
  tenant_id TEXT NOT NULL,
  person_id TEXT NOT NULL,
  thread_id TEXT NOT NULL,
  memory_item_id TEXT NOT NULL,
  relationship TEXT NOT NULL DEFAULT 'supports',
  status TEXT NOT NULL CHECK (status IN ('proposed','accepted','rejected')),
  proposed_by TEXT NOT NULL CHECK (proposed_by IN ('clinician','model','system')),
  decided_by TEXT,
  decided_at TEXT,
  created_at TEXT NOT NULL,
  UNIQUE(thread_id, memory_item_id)
);

-- 5. Model-created inference with explicit evidence
CREATE TABLE clinical_inferences (
  id TEXT PRIMARY KEY,
  tenant_id TEXT NOT NULL,
  person_id TEXT NOT NULL,
  inference_type TEXT NOT NULL,
  display_text TEXT NOT NULL,
  status TEXT NOT NULL CHECK (
    status IN ('proposed','accepted','dismissed','expired')
  ),
  ai_inference_id TEXT,
  created_at TEXT NOT NULL,
  expires_at TEXT,
  decided_by TEXT,
  decided_at TEXT
);

CREATE TABLE clinical_inference_evidence (
  inference_id TEXT NOT NULL,
  evidence_type TEXT NOT NULL,
  evidence_id TEXT NOT NULL,
  rank INTEGER NOT NULL,
  PRIMARY KEY(inference_id, evidence_type, evidence_id)
);

-- 6. Embedding metadata. Vector itself can live in provider-specific storage.
CREATE TABLE clinical_retrieval_documents (
  id TEXT PRIMARY KEY,
  tenant_id TEXT NOT NULL,
  person_id TEXT NOT NULL,
  source_type TEXT NOT NULL,
  source_id TEXT NOT NULL,
  text_for_retrieval TEXT NOT NULL,
  content_hash TEXT NOT NULL,
  embedding_model TEXT,
  embedding_ref TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  UNIQUE(tenant_id, source_type, source_id)
);

```

6.1 Required indexes

```

CREATE INDEX idx_thoughts_person_time
  ON clinician_thoughts(tenant_id, person_id, recorded_at DESC);

CREATE INDEX idx_memory_person_type_status
  ON clinical_memory_items(tenant_id, person_id, item_type, status);

CREATE INDEX idx_memory_person_label
  ON clinical_memory_items(tenant_id, person_id, normalized_label);

CREATE INDEX idx_threads_person_status
  ON clinical_threads(tenant_id, person_id, status, last_seen_at DESC);

CREATE INDEX idx_memberships_thread_status
  ON clinical_thread_memberships(tenant_id, thread_id, status);

CREATE INDEX idx_inference_person_status
  ON clinical_inferences(tenant_id, person_id, status, created_at DESC);

CREATE INDEX idx_retrieval_person
  ON clinical_retrieval_documents(tenant_id, person_id, source_type);

```

6.2 Tenant and person rules

- Every durable patient-bound row carries `tenant_id` and `person_id`.
- Do not accept `tenant_id` from a browser payload. Resolve it from authenticated context.
- The clinician identity is the authenticated clinician person/account context, not a caller-supplied string.
- New feature repositories must require `TenantContext` even while older call sites still use direct queries.
- Mirror every table into Postgres with RLS policy generation when Postgres schema is updated.
- Do not put raw PHI text in events when a governance-zone rule says protected content stays outside the event payload. Events can point to protected source records by ID and store hashes and typed metadata.

7. Domain Events and Payload Contracts

Event	When	Payload minimum
clinician_thought.recorded.v1	Audio capture finalized	{thoughtId, personId, durationMs, sourceSessionId?, recordedAt}
clinician_thought.transcribed.v1	Transcript created	{thoughtId, transcriptId, version, transcriptHash, provider, model}
clinician_thought.extraction_completed.v1	Candidate item set produced	{thoughtId, transcriptId, itemIds[], taskVersion, aiInferenceId}
clinical_memory.item_approved.v1	Clinician accepts item	{memoryItemId, thoughtId, itemType, statementClass, approvedBy}
clinical_memory.item_rejected.v1	Clinician removes candidate	{memoryItemId, thoughtId, rejectedBy}
clinical_memory.item_corrected.v1	Approved item corrected by clinician	{priorItemId, replacementItemId, reason?, correctedBy}
clinical_thread.created.v1	New longitudinal thread created	{threadId, threadType, canonicalLabel, createdBy}
clinical_thread.connection_proposed.v1	Model or clinician suggests membership	{membershipId, threadId, memoryItemId, proposedBy, score?}
clinical_thread.connection_accepted.v1	Clinician accepts candidate connection	{membershipId, decidedBy}
clinical_thread.connection_rejected.v1	Clinician rejects candidate connection	{membershipId, decidedBy}
clinical_inference.proposed.v1	Model suggests a pattern/connection	{inferenceId, inferenceType, evidences[], aiInferenceId}
clinical_inference.accepted.v1	Clinician explicitly accepts inference	{inferenceId, decidedBy}
clinical_inference.dismissed.v1	Clinician dismisses inference	{inferenceId, decidedBy}
session_prep.generated.v1	Prep generated for display	{prepId, evidences[], aiInferenceId, generatedAt}
clinician_patient_query.answered.v1	Ask Steady answer generated	{queryId, evidences[], aiInferenceId, generatedAt}

All events inherit the existing longitudinal envelope: tenant, person, actor, occurred/recorded timestamps, payload version, correlation ID, source system, and provenance. New payload schemas belong in the same registry/replay discipline as existing spine events.

8. State Machines

8.1 Thought capture state

```

capturing
  | Done
  v
processing
  | transcript + extraction succeed
  v
review
  | Save Thoughts
  v
saved

processing -- transcript succeeds, extraction fails --> review_transcript_only
processing -- unrecoverable upload/transcript failure --> failed
capturing -- Cancel --> discarded
review -- Discard --> discarded

```

Rules

- Saving is idempotent. A repeated Save Thoughts command with the same idempotency key must not duplicate approved items.
- A processing failure never deletes captured audio or a completed transcript unless policy explicitly requires deletion.

- A discarded thought stays represented in audit/event history, but its protected content follows retention policy.
- Once an approved memory item exists, edits create a replacement that supersedes the prior item. Do not mutate history in place.

9. AI Gateway Task Contracts

Do not call Anthropic, OpenAI, or any other provider directly from this feature. Register tasks under the gateway architecture defined by ADR 0012. If the gateway has not landed when implementation begins, Phase 1 may ship transcription behind a provider-neutral interface, but structured model tasks must wait for or land with the gateway.

Task ID	Purpose	Output	Human gate
clinician.thought.extract	Turn transcript into candidate memory items	Strict JSON extraction object	All items reviewed before approval
clinician.thread.match	Rank existing thread candidates for approved item	Thread candidates with evidence and scores	Connection requires clinician action in v1
clinician.session_prep.compose	Create concise pre-session brief from authorized evidence	Sectioned prep with evidence IDs per claim	No fact promotion, source drill-down required
clinician.patient_ask.answer	Answer a patient-scoped clinician query from retrieved evidence	Answer + claim/evidence map	Read-only
clinician.note.draft_from_thoughts	Draft formal note text from clinician-selected approved items	Draft prose + source item IDs	Clinician must review/sign

9.1 Extraction JSON contract

```
type ThoughtExtractionV1 = {
  schemaVersion: "1";
  transcriptId: string;
  items: Array<{
    tempId: string;
    itemType:
      | "observation" | "symptom" | "person_relationship" | "event"
      | "intervention" | "intervention_response" | "treatment_target"
      | "follow_up" | "goal" | "theme"
      | "clinician_hypothesis" | "clinician_uncertainty";
    statementClass:
      | "clinician_observation" | "patient_report"
      | "clinician_hypothesis" | "clinician_uncertainty";
    displayText: string;
    normalizedLabel: string | null;
    sourceStart: number | null;
    sourceEnd: number | null;
    sourceQuoteHash: string | null;
    numericFacts?: Array<{
      name: string;
      value: number;
      unit?: string;
    }>;
  }>;
};
```

9.2 Extraction system rules

- Extract only what is supported by the transcript.
- Do not diagnose.
- Do not turn clinician speculation into patient fact.
- Preserve negation and uncertainty.
- Separate patient report from clinician observation.
- Keep numeric values exactly as stated and mark approximations when the transcript says 'about' or similar.
- Do not create thread relationships in the extraction task. Thread matching is a separate task after item review.
- Return source offsets when reliable. If offsets cannot be guaranteed, return null rather than inventing them.

10. Hybrid Retrieval Design

Ask Steady and thread matching should not use embeddings alone. First enforce authorization and patient scope, then retrieve from structured and narrative sources. The ranking layer can combine lexical, structured, recency, and semantic signals.

```
authorized candidates =  
  records WHERE tenant_id = ctx.tenantId  
            AND person_id = requestedPersonId  
            AND zone/purpose is allowed
```

```
score =  
  0.35 * semantic_similarity  
+ 0.25 * lexical_match  
+ 0.20 * structured_concept_match  
+ 0.10 * recency_signal  
+ 0.10 * source_reliability_weight
```

```
Then:  
- deduplicate near-identical sources  
- cap evidence per source type  
- require evidence IDs for every generated claim  
- never let similarity itself create a clinical relationship
```

Initial weights above are implementation defaults, not clinical truth. Put them behind versioned retrieval policy so evaluation can change them without changing the meaning of stored records.

- For thread matching, prefer existing canonical label + accepted members + recent evidence.
- For Ask Steady, retrieve approved clinical memory, formal notes the user may access, relevant check-ins/assessments, and selected patient activity according to purpose.
- Exclude rejected candidates, dismissed inferences, expired temporary intelligence, and unauthorized governance zones.
- An answer must be able to show exactly which records supported each major statement.

11. Session Prep Pipeline

```
OPEN PATIENT
|
+-- Load last reviewed/session context
+-- Load open clinician follow-ups
+-- Load active threads
+-- Load approved memory since last session
+-- Load permitted between-visit signals
+-- Load active clinician-accepted hypotheses
|
+-- deterministic assembly of facts
|
+-- optional gateway composition task
|
+-- claim validation against evidence IDs
|
+-- render Session Prep
```

- Keep Session Prep short enough to scan in about one minute.
- Recommended sections: Last Session, You Wanted to Revisit, Between-Visit Changes, Active Threads, Steady Noticed.
- The first four sections should be mostly deterministic where data allows. Use generation mainly for concise wording and synthesis.
- Steady Noticed is explicitly machine-created. Every item gets a Why am I seeing this? action that opens evidence.
- Withhold any generated claim whose citations cannot be resolved to authorized source records.
- Cache only if the cache key includes person, tenant, evidence cutoff, retrieval/prompt version, and relevant policy versions.

12. Ask Steady About This Patient

- Place a patient-scoped query box in the patient workspace or Thoughts/Threads view.
- Queries such as 'When did sleep start getting worse?', 'What have I written about her sister?', and 'How has grounding been working?' should work without special syntax.
- Every answer is read-only. It may surface clinician-created follow-ups or offer a link to create one, but it does not write clinical memory from the answer itself.
- Use patient/tenant scope before retrieval, not after generation.
- Return a source list and claim-to-source mapping. The UI should offer View source.
- If evidence is insufficient or conflicting, say so. Do not smooth conflicting history into one answer.

13. Audio and Transcription Lifecycle

Stage	Required behavior	Failure behavior
Capture	Browser MediaRecorder or equivalent, patient context visible, local temporary buffer	Keep unsent buffer if safe and technically possible
Upload	Authenticated, patient-scoped upload token or server route; content-type and size checks	Retry with idempotency key
Storage	Encrypted object storage, tenant/person metadata outside object name where practical	Never expose public object URL
Transcription	Provider-neutral service interface; BAA-approved provider in real deployment	If provider fails, keep audio and mark retryable
Verification	Hash transcript; store provider/model/version and confidence metadata	Low-confidence spans shown to clinician
Retention	Org policy: retain, delete after verified transcript, or bounded retention	Deletion event + timestamp; do not claim deletion before object is removed

Storage interface: Keep object storage behind a Steady service abstraction. Do not encode one vendor's bucket API into clinical domain code.

14. API and Command Surface

Endpoint	Command/read	Auth boundary
POST /api/clinician/member/:id/thoughts	Create thought capture, return thoughtId + upload instructions	clinician, patient scope
POST /api/clinician/member/:id/thoughts/:tid/finalize	Finalize upload and queue/process transcript	clinician, owner or permitted team member
GET /api/clinician/member/:id/thoughts/:tid	Get processing/review state	clinician, patient scope
PATCH /api/clinician/member/:id/thoughts/:tid/transcript	Create clinician-corrected transcript version	clinician, patient scope
POST /api/clinician/member/:id/thoughts/:tid/save	Approve/edit/reject extraction set atomically	clinician, patient scope
GET /api/clinician/member/:id/thoughts	List saved thoughts and status	clinician, patient scope
GET /api/clinician/member/:id/threads	List active/resolved threads	clinician, patient scope
POST /api/clinician/member/:id/threads/:threadId/connect	Accept candidate membership	clinician, patient scope
POST /api/clinician/member/:id/threads/:threadId/reject	Reject candidate membership	clinician, patient scope
GET /api/clinician/member/:id/session-prep	Build or retrieve evidence-bound prep	clinician, patient scope
POST /api/clinician/member/:id/ask	Patient-scoped retrieval + answer	clinician, patient scope

14.1 Save Thoughts request

```
{
  "thoughtId": "01...",
  "transcriptVersion": 2,
  "idempotencyKey": "uuid-or-ulid",
  "items": [
    {
      "candidateId": "tmp-1",
      "decision": "approve",
      "itemType": "follow_up",
      "statementClass": "clinician_observation",
      "displayText": "Check sleep next session.",
      "normalizedLabel": "sleep"
    },
    {
      "candidateId": "tmp-2",
      "decision": "reject"
    }
  ],
  "threadDecisions": [
    {
      "candidateMembershipId": "01...",
      "decision": "accept"
    }
  ]
}
```

The server validates every approved item against the current transcript/extraction generation. A stale browser submission must return a conflict rather than writing against an older transcript.

15. Permissions Matrix

Action	Assigned clinician	Other care-team clinician	Supervisor	Org admin
Record thought	Yes	Only if patient access permits	Only if patient access permits	No by default
Hear retained audio	Yes	Policy-dependent	Policy-dependent	No by default
Edit own transcript before save	Yes	No	No	No
Approve own extracted items	Yes	No	No	No

Action	Assigned clinician	Other care-team clinician	Supervisor	Org admin
View saved clinical memory	Yes	If patient access permits	If supervision scope permits	No unless clinical role
Correct approved memory	Yes, with provenance	If patient access permits and policy allows	Yes within scope	No unless clinical role
Accept thread connection	Yes	If clinical access permits	Yes within scope	No unless clinical role
Ask Steady	Yes	If clinical access permits	Yes within scope	No unless clinical role
Configure retention	No	No	No	Yes, policy surface

Use the repository's evolving role-assignment model rather than hard-coding the current single `users.role` field into new feature authorization.

16. Corrections, Deletion and Supersession

- Before save, transcript edits create a new transcript version. Extraction reruns against the latest version.
- After approval, clinical memory corrections create a replacement item with `supersedes_id` and an event that references both IDs.
- Rejecting a candidate before approval is not the same as deleting a clinical record. Keep enough audit metadata to show the candidate was reviewed without retaining unnecessary discarded PHI longer than policy permits.
- Dismissed model inferences remain in the AI/provenance ledger according to evaluation policy, but must not appear as active clinical truth.
- Audio deletion follows retention policy and records deletion completion. It does not delete the transcript or approved memory unless a separate authorized process requires that.
- Formal-note content follows formal record correction rules. A private clinician thought should not silently become a formal note merely because an AI draft used it.

17. Screen-Level UX Specification

17.1 Patient overview

```

■ Sarah M.                                Current state / consent / owner
■
■ Since your last review                    [Record Thoughts]
■ Evidence-backed summary ...
■
■ Session Prep
■ Last session ...
■ You wanted to revisit ...
■ Between-visit changes ...
■ Steady noticed ...                      [Why am I seeing this?]
■
■ Active work / gates / existing sections

```

- Place Record Thoughts in the existing action rail/header region so it is available without scrolling into the formal record.
- Do not replace the existing 'Since your last review' evidence-backed section. Session Prep sits near it and reuses evidence/review conventions.
- Use the existing ReviewBadge vocabulary or an equivalent explicit label for generated/validated/reviewed states.

17.2 Thoughts view

```
THOUGHTS
[ Record Thoughts ]

Sep 3, 2:14 PM    Saved
  0:58 audio      Transcript available
  6 approved items
  1 connection accepted
  1 connection rejected

Observation      Calmer presentation today
Follow-up         Check sleep next session
Hypothesis       Possible abandonment connection [Unconfirmed]
...
[Open source] [Use selected in clinical note]
```

17.3 Threads view

```

THREADS

Sleep disruption                                Active
  Sep 03 Clinician thought: sleep still poor
  Aug 26 Patient report: improved
  Aug 12 Clinician note: difficulty falling asleep

Grounding                                       Active
  Sep 03 Distress about 7 -> 3 during session
  Aug 24 Patient reported using twice
  Aug 17 Introduced

[Open thread] -> chronological evidence, source type, review state

```

17.4 Processing and failure copy

State	User-facing copy direction
Uploading	Saving your recording...
Transcribing	Creating a transcript...
Organizing	Organizing your thoughts...
Transcript ready, extraction failed	Your transcript is safe. Steady could not organize it yet. You can save the transcript now or retry organization.

State	User-facing copy direction
Low confidence	A few words may need review. Check the highlighted transcript before saving.
Stale edit conflict	This transcript changed in another tab or session. Reload the latest version before saving.
Permission lost	Your access to this patient changed. The recording was not added to the patient's clinical memory.

18. Security, Privacy and Audit

- Treat audio, transcripts, embeddings, generated prep, query prompts, and query answers as sensitive patient data.
- Do not log raw transcript text, patient query text, or model prompts in general application logs.
- AI gateway provenance may store protected input references and hashes rather than duplicating full PHI when not required.
- Encrypt stored audio and protected clinical text according to the same key management strategy used for other protected content.
- Audit creation, transcript correction, approval, correction/supersession, thread decisions, formal-note transfer, audio deletion, and privileged playback where policy requires it.
- Patient-scoped semantic search must be tested for cross-tenant and cross-patient leakage through direct IDs, lexical search, vector lookup, aggregate counts, and generated context.
- No external analytics event should contain patient text, transcript content, thread labels that reveal clinical content, or query text.

19. Test and Evaluation Matrix

Area	Required tests
Domain	State transitions, item approval/rejection, supersession, thread membership, inference lifecycle, idempotent save
Database	Schema boot on fresh and existing SQLite DB, indexes, FK integrity, tenant columns, Postgres schema parity
Event spine	Every mutation appends expected event, replay/projection parity, point-in-time reconstruction, no direct orphan writes
Tenancy	Foreign tenant person ID returns not-found, foreign write refused, search and retrieval cannot enumerate another tenant
Authorization	Unassigned clinician, expired access, supervisor scope, org admin without clinical role
Audio	Permission denied, interrupted upload, retry, duplicate finalize, unsupported type, size limit, retention deletion
Transcription	Provider timeout, malformed result, low-confidence spans, corrected transcript versioning
Extraction	Negation, uncertainty, patient quote, clinician hypothesis, multiple people, numeric ratings, conflicting claims
Thread matching	Good candidate ranked, unrelated candidate rejected, no auto-link, rejected relationship stays rejected
Session Prep	Open follow-ups present, between-visit data cutoff, every generated claim has resolvable evidence, unauthorized evidence excluded
Ask Steady	Patient scope, conflicting evidence, insufficient evidence, source links, prompt injection in stored patient text
Formal note bridge	Only selected approved items included, source IDs preserved, draft never auto-signed

19.1 Minimum AI golden set

- At least 100 short clinician utterances before pilot, with a heavy share of ambiguity, negation, uncertainty, and quoted patient speech.
- Score item-type accuracy, statement-class accuracy, source grounding, numeric preservation, missed follow-ups, unsupported additions, and unsafe fact promotion.
- Unsafe fact promotion is a release blocker. Examples include converting 'might be drinking more' to 'increased alcohol use' or 'not sure she is ready' to 'patient not ready.'
- Keep the evaluation set versioned by AI task and run it in CI without requiring live model calls for every test.

20. File-Level Implementation Map

Path	Action
src/app/clinician/member/[id]/page.tsx	Add Record Thoughts entry point and Session Prep placement. Keep page composition thin.
src/app/clinician/member/[id]/thoughts/page.tsx	NEW. Thought history, source drill-down, capture/review launcher.
src/app/clinician/member/[id]/threads/page.tsx	NEW. Active/resolved threads and chronological evidence.
src/app/api/clinician/member/[id]/thoughts/...	NEW. Capture, finalize, review/save routes or equivalent command endpoints.
src/components/clinical/RecordThoughtsButton.tsx	NEW. Launch action.
src/components/clinical/ThoughtRecorder.tsx	NEW client component. Media capture only, no clinical domain decisions.
src/components/clinical/ThoughtReview.tsx	NEW. Transcript + organized item review.
src/components/clinical/SessionPrep.tsx	NEW. Evidence-bound prep presentation.
src/components/clinical/ThreadList.tsx	NEW. Thread overview.
src/lib/clinical/thoughts.ts	NEW. Clinical thought use cases and read model.
src/lib/clinical/memory.ts	NEW. Approval, correction, memory item mapping.
src/lib/clinical/threads.ts	NEW. Thread creation/membership/query.

Path	Action
src/lib/clinical/session-prep.ts	NEW. Deterministic prep assembly and optional gateway composition.
src/lib/clinical/patient-query.ts	NEW. Authorized retrieval orchestration.
src/lib/clinical/retrieval.ts	NEW. Hybrid retrieval policy and evidence packaging.
src/lib/spine.ts	ADD registered event helpers or domain recorder calls for this feature.
src/lib/projections.ts	ADD projectors for approved memory, thread state, and follow-up/read models as needed.
src/lib/db.ts	ADD SQLite schema/migrations and tenant-scoped tables.
scripts/pg-schema.sql	MIRROR new tables and RLS coverage.
src/lib/repository.ts	ADD typed repositories/table registrations. New feature must not bypass TenantContext.
src/lib/ai-gateway/...	REGISTER extraction, thread match, prep, query, note draft tasks when gateway lands.
tests/clinician-thoughts*.test.ts	NEW domain, route, tenancy, AI fixtures, replay tests.
tests/tenant-isolation.test.ts	ADD attack cases for every new table/read path.
docs/adr/0014-clinician-thoughts-memory.md	NEW ADR recommended before merge, recording source/memory/inference boundaries.

21. Numbered Build Sequence

Phase 0 - Architecture gate

Build

- Confirm ADR 0012 gateway implementation status.
- Add/approve feature ADR.
- Register new tenant-scoped tables in schema guards.
- Decide audio retention defaults for demo and production.

Definition of done

- No code path can accidentally call a model provider directly.
- Schema plan is mirrored for SQLite and Postgres.
- Feature flag names exist.

Phase 1 - Capture and transcript

Build

- Add tables for thoughts and transcript versions.
- Add recorder UI and secure upload/storage abstraction.
- Add transcription service interface.
- Implement thought state machine and source events.
- Add Thoughts history view.

Definition of done

- Clinician records, receives transcript, corrects transcript, and saves source thought.
- Interrupted processing does not lose a completed transcript.
- Cross-tenant access tests pass.

Phase 2 - Structured organization

Build

- Register extraction task.
- Create candidate memory items.
- Build review cards and Save Thoughts atomic command.
- Add approved memory projection and supersession rules.

Definition of done

- Uncertainty is preserved.
- No candidate becomes approved without clinician save.
- Replay reproduces approved memory state.

Phase 3 - Threads and follow-ups

Build

- Create thread/membership model.
- Add thread-match task.
- Require Connect / Not related decision.
- Build thread timeline.
- Feed approved follow-ups into patient work context.

Definition of done

- No auto-link in v1.
- Rejected links remain rejected unless a clinician deliberately revisits them.
- Thread evidence always opens source.

Phase 4 - Session Prep

Build

- Assemble deterministic prep inputs.
- Register optional compose task.
- Validate every generated claim against evidence.
- Render Session Prep near patient overview.

Definition of done

- Open follow-ups and active threads appear.
- Steady Noticed is visually distinct.
- Uncited generated claims are withheld.

Phase 5 - Ask Steady

Build

- Create retrieval documents and embedding abstraction.
- Implement authorized hybrid retrieval.
- Register patient query task.
- Add source drill-down and conflicting-evidence behavior.

Definition of done

- No cross-patient or cross-tenant retrieval.
- Every answer returns evidence.
- Insufficient evidence produces an explicit limitation, not a fabricated answer.

Phase 6 - Formal note bridge and rollout

Build

- Allow clinician-selected approved items to feed a note draft.
- Preserve source item IDs.
- Add admin retention policy controls.
- Add telemetry and pilot flags.

Definition of done

- Draft never auto-signs.
- Private thought does not become formal note without clinician action.
- Pilot can be disabled per tenant without data loss.

22. Feature Flags and Rollout

CLINICIAN_THOUGHTS_CAPTURE
CLINICIAN_THOUGHTS_EXTRACTION
CLINICIAN_THREADS
CLINICIAN_SESSION_PREP
CLINICIAN_PATIENT_ASK
CLINICIAN_NOTE_BRIDGE

- Flags should be tenant-aware where the current feature-flag system allows it.
- Capture can launch before semantic retrieval.
- Do not expose a disabled downstream surface just because data for it already exists.
- A flag change must not delete or rewrite previously stored patient history.

23. Product and Quality Telemetry

Metric	Why track it	PHI rule
recording_duration_ms	Capture friction and behavior	No audio/text in analytics
processing_latency_ms by stage	Operational performance	IDs may be pseudonymous/internal only
transcript_correction_rate	Transcription quality signal	No corrected text in analytics
item_edit_rate / rejection_rate	Extraction quality signal	No item text
thread_acceptance_rate	Link quality signal	No thread labels
session_prep_open_rate	Workflow use	No patient content
source_drilldown_rate	Trust/evidence behavior	No source content
patient_query_grounding_failure_rate	Retrieval/answer quality	No query text

24. Coding-Agent Execution Contract

- Work in the numbered phase order. Do not jump to semantic retrieval before capture, tenancy, provenance, and approval semantics are working.
- At the start of each phase, inspect the current branch because ADR 0010, ADR 0011, and ADR 0012 are active migration areas.
- Reuse current Steady components and design tokens. Do not create a second clinician visual language.
- Use the event spine for every durable domain change. Do not add direct-only writes.
- Use TenantContext/repository abstractions for new feature data even if neighboring legacy code still performs raw queries.
- Route model work through the AI Gateway. If the gateway is unavailable, do not silently add a direct provider integration.
- Keep raw protected content out of ordinary logs and analytics.
- Add tests before moving to the next phase. A green build alone is not enough. The phase Definition of Done must pass.
- Never alter patient safety gates or clinical permissions as a side effect of this feature.
- When a requirement conflicts with a newer accepted ADR on the target branch, stop that subtask, document the conflict in the PR, and follow the newer architecture decision rather than guessing.

Merge rule: Each phase should be independently mergeable behind feature flags. Avoid one giant PR that combines audio, AI, retrieval, and formal-note behavior.

25. Release Acceptance Checklist

- ■ Record Thoughts is available from the correct patient context and displays patient identity.
- ■ Audio capture, upload, transcription, and failure recovery work without requiring clinicians to classify notes first.
- ■ Transcript is visible and editable before memory approval.
- ■ Structured items preserve patient report, clinician observation, hypothesis, and uncertainty as separate meanings.
- ■ Approve all / edit / remove workflow is fast and atomic.
- ■ No model-proposed thread relationship becomes accepted without clinician action in v1.
- ■ Approved memory is source-backed and correctable without erasing prior history.
- ■ Thoughts, Threads, Session Prep, and Ask Steady obey tenant + patient authorization.
- ■ Every generated Session Prep claim and Ask Steady claim can open its evidence.
- ■ Formal clinical notes remain separate from private clinician thoughts.
- ■ AI provider calls are behind the gateway architecture.
- ■ Cross-tenant and cross-patient retrieval attacks pass.
- ■ Event replay/projection tests pass for all new durable clinical state.
- ■ Audio retention/deletion policy is implemented and testable.
- ■ No patient text is emitted to normal logs or external analytics.
- ■ Feature can be disabled per rollout plan without corrupting stored history.

Appendix A. Prompt Contract Skeletons

A.1 clinician.thought.extract

SYSTEM PURPOSE:
Organize a clinician's spoken thought into typed candidate items.

BOUNDARIES:

- Use only the supplied transcript.
- Do not diagnose.
- Do not infer facts that were not said.
- Preserve who said what.
- Preserve uncertainty and negation.
- Return only schema-valid JSON.
- Do not link to prior threads in this task.

INPUT:
tenant-scoped person context ID, transcript ID, transcript text

OUTPUT:
ThoughtExtractionV1

A.2 clinician.session_prep.compose

SYSTEM PURPOSE:
Render a short clinician-facing preparation brief from supplied evidence.

BOUNDARIES:

- Every claim must cite one or more supplied evidence IDs.
- Do not add diagnoses, treatment recommendations, or unsupported causal language.
- Keep model-created synthesis in the Steady Noticed section.
- If evidence conflicts, state the conflict.
- If a claim cannot be grounded, omit it.

OUTPUT:

```
{
  "lastSession": [{ "text": "...", "evidenceIds": ["..."] }],
  "revisit": [{ "text": "...", "evidenceIds": ["..."] }],
  "betweenVisit": [{ "text": "...", "evidenceIds": ["..."] }],
  "activeThreads": [{ "text": "...", "evidenceIds": ["..."] }],
  "steadyNoticed": [{ "text": "...", "evidenceIds": ["..."] }]
```

A.3 clinician.patient_ask.answer

SYSTEM PURPOSE:
Answer a clinician's question about one authorized patient from retrieved evidence.

BOUNDARIES:

- Do not use knowledge outside the supplied evidence for patient-specific claims.
- Cite evidence IDs for each major claim.
- Distinguish patient report, clinician observation, and model synthesis.
- Say when evidence is insufficient or conflicting.
- Do not write or change clinical memory.

OUTPUT:

```
{
  "answer": "...",
  "claims": [
    { "text": "...", "evidenceIds": ["..."], "kind": "source_fact|synthesis" }
  ]
}
```

Appendix B. Repository Review Notes

- The reviewed clinician member page already follows a 'first thirty seconds' design, with identity, current state, evidence-backed summary, active work, gates, and source links. This feature should sit inside that mental model.
- The reviewed repository uses better-sqlite3 today. Any specification that assumes live Postgres RLS would overstate the current control. Postgres schema and RLS are migration targets, not the running runtime on this branch.

- ADR 0010 states event dual-write and replay verification are shipped, but event-only writes are still held for a later cutover.
- ADR 0011 states tenant/person building blocks and RLS tests exist, while end-to-end product call-site migration is still incomplete.
- ADR 0012 requires a single AI Gateway with versioned tasks, structured outputs, PHI policy, provenance, retries, and evaluation. The clinician-thought feature should become one of the first consumers built entirely to that contract.

Appendix C. Build Outcome

When this handoff is fully implemented, a clinician can speak naturally after a session, review Steady's organization, save approved clinical memory, see longitudinal threads, arrive at the next visit with an evidence-backed Session Prep, and ask questions across the patient's authorized history. The feature remains inspectable because every meaningful item points back to source, every model conclusion stays visibly distinct from fact, and every accepted relationship is attributable to a human decision.

STEADY ENGINEERING SPECIFICATION v2.1

Binding Amendment: Full Longitudinal Session Prep and Source Rules

Status: This amendment is part of the coding handoff. Where it adds specificity to v2.0, these rules control implementation.

26. Session Prep Scope, Explicit Data Sources

Session Prep is not a summary of the previous clinician note. It is a patient-scoped, longitudinal preparation layer that assembles the most relevant authorized evidence available to the clinician before the next encounter.

Required input classes

- **Previous formal session notes:** prior clinician documentation, including the immediately preceding session and older notes when retrieval determines they are relevant.
- **Clinician Thoughts:** approved structured memory from recorded clinician thoughts, open follow-ups, clinician hypotheses, clinician uncertainty, accepted themes, and accepted thread relationships.
- **Check-ins:** between-visit patient check-ins, symptom/distress values, reported changes, repeated patterns, and meaningful deviations from baseline.
- **AI Companion interactions:** clinically relevant patient statements, approved/persisted Companion memory, recurring subjects, changes, and safety-relevant interaction metadata that the clinician is authorized to see.
- **Assessments and measures:** scores, score changes, completed instruments, relevant thresholds, and longitudinal movement.
- **Practice/intervention activity:** completed exercises, grounding or other intervention use, response data, adherence/activity patterns, and patient-reported benefit or difficulty.
- **Treatment and care context:** active goals, targets, modules, clinician review items, unresolved gates, referrals, care-plan context, and other authorized longitudinal events.
- **Safety information:** only the safety data appropriate for clinician display under Steady's existing safety and governance rules. Session Prep must never weaken or replace the dedicated safety engine.

Source inclusion is permission-bound, not merely availability-bound. The existence of data in Steady does not automatically make it eligible for Session Prep. Tenant, patient, clinician authorization, consent, governance zone, purpose, retention state, and source-specific visibility rules are applied before retrieval or generation.

27. Companion Interaction Rules

AI Companion activity must be a first-class Session Prep input, but it requires stronger source semantics than ordinary note retrieval.

- A patient's statement to the Companion remains **patient-reported evidence**. It does not become clinician-observed fact.
- A Companion-generated response is **not clinical evidence about the patient** merely because the model said it.
- A model-created Companion memory item remains model-mediated or patient-derived according to its provenance. Do not relabel it as clinician-verified.

- Session Prep may summarize repeated Companion subjects or changes only when the underlying authorized interactions support the statement.
- Do not surface every Companion conversation. Retrieve for clinical relevance, change, open treatment threads, clinician follow-ups, safety context, and recency.
- Sensitive Companion material must obey any product rule that limits clinician visibility. Session Prep cannot be used to bypass a visibility boundary.
- If the patient corrects or retracts a Companion memory, retrieval must use the current/superseding state while preserving historical provenance.
- Companion content that contains prompt injection or instructions to the model is treated as untrusted patient text, never as system instruction.

28. Full Session Prep Assembly

```

SESSION PREP REQUEST
|
+-- Resolve tenant + clinician + patient authorization
+-- Establish evidence cutoff / appointment context
|
+-- Prior formal session notes
+-- Approved clinician thoughts + open follow-ups
+-- Active longitudinal threads
+-- Check-ins since last encounter + meaningful older trend context
+-- AI Companion interactions / approved memory eligible for clinician view
+-- Assessments + measures
+-- Practice / intervention activity
+-- Treatment targets / goals / care context
+-- Safety context permitted for this surface
|
+-- Normalize all sources into typed evidence objects
+-- Deduplicate / supersession resolution
+-- Detect change from prior session and baseline
+-- Retrieve older evidence when needed to explain current thread
|
+-- Deterministic facts first
+-- Optional AI synthesis through AI Gateway
+-- Claim-to-evidence validation
|
+-- Render clinician prep

```

Session Prep must be longitudinal. The immediately previous session provides an anchor, not a retrieval boundary. If a current issue connects to something from three months earlier, Steady should be able to retrieve that history and present it when relevant.

29. Session Prep Presentation Contract

- **Last Session:** the main clinical context from the prior encounter, including unresolved items.
- **You Wanted to Revisit:** open clinician follow-ups, explicit reminders, unresolved hypotheses, and items carried forward from clinician thoughts or formal notes.
- **Since the Last Session:** meaningful check-in changes, assessments, intervention activity, and relevant patient-reported developments.
- **Companion Context:** clinically relevant between-visit themes or statements from AI Companion interactions, clearly labeled by source.
- **Active Threads:** current longitudinal themes, symptoms, people/relationships, treatment targets, goals, interventions, and response trajectories.

- **Changes and Trends:** meaningful movement over time, not just raw values. Include baseline comparison when available and clinically appropriate.
- **Steady Noticed:** model-generated patterns, candidate connections, or synthesis. This section must remain visually distinct from sourced facts.
- **Safety / Attention Needed:** when permitted and appropriate, surface existing safety or care-attention information without replacing the dedicated safety workflow.

The screen should be broad in evidence coverage but concise in presentation. The goal is not to show everything Steady knows. The goal is to give the clinician the most useful preparation for this session, with one-click access to the evidence behind it.

30. Evidence Object Contract

```
type SessionPrepEvidence = {
  evidenceId: string;
  tenantId: string;
  personId: string;
  sourceType:
    | "formal_session_note"
    | "clinician_thought"
    | "clinical_memory_item"
    | "checkin"
    | "companion_patient_statement"
    | "companion_memory"
    | "assessment"
    | "measure"
    | "practice_activity"
    | "intervention_response"
    | "treatment_goal"
    | "clinical_thread"
    | "safety_context"
    | "other_authorized_event";
  sourceId: string;
  occurredAt: string;
  recordedAt: string;
  provenanceClass:
    | "clinician_documented"
    | "clinician_observed"
    | "patient_reported"
    | "system_measured"
    | "model_derived";
  reviewState?: "unreviewed" | "reviewed" | "approved" | "corrected";
  supersedesId?: string | null;
  displayText: string;
  structuredFacts?: Record<string, unknown>;
};
```

The generation layer should receive normalized evidence objects rather than arbitrary database rows. This makes source class and provenance visible to both deterministic assembly and the model.

31. New Non-Negotiable Session Prep Rules

- **No source flattening.** Formal clinician documentation, patient check-ins, Companion statements, system measurements, and model synthesis must retain different provenance classes.
- **No previous-note tunnel vision.** Session Prep must look across authorized longitudinal history when current context requires it.
- **No Companion-as-clinician.** Patient statements in Companion interactions remain patient-reported. Companion output never becomes clinician observation.
- **No inference promotion.** A Steady pattern or correlation remains model-derived unless the clinician explicitly accepts it through a defined workflow.

- **No unsupported causal language.** 'Sleep worsened during two periods of family conflict' may be supported. 'Family conflict caused the sleep problem' requires stronger evidence and should not be generated by default.
- **No silent contradiction resolution.** If sources disagree, Session Prep should represent the disagreement or prefer a corrected/superseding authoritative source according to provenance rules.
- **No future-data leakage.** Prep for a historical encounter or audit reconstruction may use only information Steady had at the relevant cutoff.
- **No raw-volume dumping.** More data is not automatically better. Rank by relevance, change, open clinical work, recency, and longitudinal importance.
- **No hidden evidence.** Every material generated claim must expose its source records through Why am I seeing this? or View source.
- **No safety substitution.** Session Prep may display authorized safety context but never becomes the safety decision engine.
- **No visibility bypass.** Retrieval cannot expose data the clinician could not otherwise access under the applicable product and consent rules.

32. Ranking and Relevance Rules

Session Prep retrieval should favor what is most likely to matter in the upcoming encounter, not simply the newest records.

```
session_prep_relevance =
  open_follow_up_weight
+ active_thread_match
+ change_from_baseline
+ change_since_last_session
+ clinician_marked_importance
+ treatment_goal_or_target_match
+ safety_attention_weight
+ semantic_relevance
+ recency
```

Then:

- resolve superseded/corrected records
- cap repetitive evidence
- preserve contradictory evidence
- retrieve older history when it explains an active current thread
- keep source/provenance attached through generation and rendering

Weights should be versioned and evaluated. Safety weighting does not authorize Session Prep to invent a new safety classification.

33. Session Prep Acceptance Criteria, Added to v2.0

- Session Prep can use prior formal session notes, not only the immediately previous note.
- Session Prep includes approved clinician Thoughts and open follow-ups.
- Session Prep consumes relevant check-ins and longitudinal check-in trends.
- Session Prep consumes authorized clinically relevant AI Companion interactions and Companion memory.
- Companion patient statements are visibly identifiable as patient-reported evidence.
- Session Prep can use assessments, measures, practices/interventions, treatment goals/targets, and permitted safety context.
- The system can retrieve older history when needed to explain an active thread.
- Every generated material claim maps to one or more authorized evidence IDs.
- A clinician can drill from a summary statement to its underlying source.

- Conflicting sources are not silently merged into a false single fact.
- Corrected/superseded information is handled according to provenance and point-in-time rules.
- A model-generated pattern remains in Steady Noticed or another explicitly model-derived state until human acceptance.
- Session Prep never uses a data source solely because it exists. Authorization, consent, purpose, governance zone, and visibility rules run before retrieval.

34. Coding Changes Required by This Amendment

- Update `src/lib/clinical/session-prep.ts` to treat AI Companion interactions and memory as explicit input adapters, alongside session notes, clinician thoughts, check-ins, assessments, and intervention activity.
- Add a normalized `SessionPrepEvidence` representation and `source/provenance` class.
- Add source adapters for the current Companion/memory, check-in, assessment/measure, practice/intervention, and formal session stores.
- Add permission and visibility checks before evidence enters the retrieval candidate set.
- Update the Session Prep AI task contract so Companion evidence cannot be relabeled as clinician-observed fact.
- Add tests for Companion provenance, corrected memory, conflicting sources, older-history retrieval, consent/visibility exclusion, prompt injection inside Companion content, and point-in-time evidence cutoff.
- Update the Session Prep UI to include Companion Context when relevant and keep Steady Noticed visually separate.
- Update Phase 4 Definition of Done to require all listed evidence classes that exist in the current Steady product, rather than only generic 'between-visit data inputs.'

Final implementation intent: Session Prep should feel as if Steady reviewed the patient's authorized longitudinal story before the clinician walked into the room, while still showing the clinician exactly what came from where and refusing to blur patient report, clinician judgement, system measurement, and AI synthesis.